

CURS INTRODUCTIV · ÎNVĂȚARE AUTOMATĂ

# Gradient Descendent

*Cum învață algoritmi să găsească cea mai bună soluție*



---

O introducere accesibilă, de la intuiție la matematică și cod ·  
10 pagini · Nivel începător – intermediar

# Ce vei învăța

*Acest curs te poartă pas cu pas de la o intuiție simplă până la implementarea reală a unuia dintre cei mai importanți algoritmi din inteligența artificială.*

|                                                    |      |
|----------------------------------------------------|------|
| 1 · Intuiția: coborârea de pe munte                | p.3  |
| Problema fundamentală pe care o rezolvă algoritmul |      |
| 2 · Funcția de cost                                | p.4  |
| Cum măsurăm cât de „greșit” este un model          |      |
| 3 · Gradientul și derivata                         | p.5  |
| Busola care arată direcția de coborâre             |      |
| 4 · Regula de actualizare                          | p.6  |
| Formula centrală a algoritmului                    |      |
| 5 · Rata de învățare                               | p.7  |
| Mărimea pasului și de ce contează enorm            |      |
| 6 · Variante: Batch, Stochastic, Mini-batch        | p.8  |
| Cum scalăm algoritmul la date reale                |      |
| 7 · Implementare în Python                         | p.9  |
| De la teorie la cod care funcționează              |      |
| 8 · Capcane, sfaturi și recapitulare               | p.10 |
| Ce trebuie să reții și ce greșeli să eviți         |      |

## CUM SĂ FOLOSEȘTI CURSUL

Fiecare capitol construiește peste cel anterior. Nu sări peste analogii — ele leagă matematica de ceva concret. Formulele sunt explicate în limbaj natural imediat după ce apar.

# Intuiția: coborârea de pe munte

*Imaginează-ți că ești pe un munte, în ceață deasă, și vrei să ajungi în vale. Nu vezi nimic în jur. Ce faci?*

*Simți cu piciorul panta din jurul tău și faci un pas în direcția în care terenul coboară cel mai abrupt. Apoi te oprești, simți din nou panta, și faci încă un pas. Repeți până când terenul devine plat — ai ajuns în vale. Exact asta face **gradient descent**.*

În învățarea automată, „muntele” este o **funcție de cost**: o suprafață matematică unde înălțimea reprezintă cât de mari sunt erorile modelului nostru. Vârfurile înalte înseamnă predicții proaste; valea înseamnă predicții bune. Scopul algoritmului este să găsească cel mai jos punct posibil.

## De ce avem nevoie de acest algoritm?

Un model de învățare automată are *parametri* — numere interne pe care le poate ajusta. Un model simplu poate avea câțiva parametri; o rețea neuronală modernă poate avea miliarde. Pentru fiecare combinație de parametri, modelul produce un anumit nivel de eroare.

Întrebarea este: **ce valori ale parametrilor produc cea mai mică eroare?** Nu putem încerca toate combinațiile — ar fi astronomic de multe. Avem nevoie de o metodă inteligentă care să ne ghideze direct spre soluția bună. Gradient descent este acea metodă.

### IDEEA ESENȚIALĂ

Gradient descent este un algoritm de **optimizare**: pornește de la o soluție aleatoare și o îmbunătățește treptat, pas cu pas, mergând mereu „la vale” pe suprafața erorii, până când ajunge (aproape) la minimum ei.

## Trei ingrediente

Pentru a coborî muntele avem nevoie de trei lucruri, pe care le vom studia pe rând în capitolele următoare:

- **O hartă a înălțimii** — funcția de cost, care ne spune cât de proastă e soluția curentă (Capitolul 2).
- **O busolă** — gradientul, care ne arată în ce direcție coboară terenul cel mai abrupt (Capitolul 3).
- **O mărime a pasului** — rata de învățare, care decide cât de mult ne mișcăm la fiecare pas (Capitolul 5).

## Funcția de cost

Înainte să putem coborî, trebuie să știm ce înseamnă „mai jos”. Funcția de cost ne dă această măsură.

O **funcție de cost** (numită și funcție de pierdere sau *loss*) ia parametrii modelului și returnează un singur număr: cât de departe sunt predicțiile modelului de valorile reale. Cu cât numărul e mai mic, cu atât modelul e mai bun.

### Un exemplu concret: regresia liniară

Să zicem că vrem să prezicem prețul unei case în funcție de suprafața ei. Modelul nostru e o linie dreaptă:  $\hat{y} = w \cdot x + b$ , unde  $x$  e suprafața,  $\hat{y}$  e prețul prezis, iar  $w$  și  $b$  sunt parametrii pe care vrem să-i găsim.

Pentru fiecare casă din datele noastre comparăm prețul prezis  $\hat{y}$  cu prețul real  $y$ . Diferența  $\hat{y} - y$  este eroarea pentru acea casă. O măsură foarte folosită este **eroarea medie pătratică** (Mean Squared Error):

#### FUNCȚIA DE COST — EROAREA MEDIE PĂTRATICĂ

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

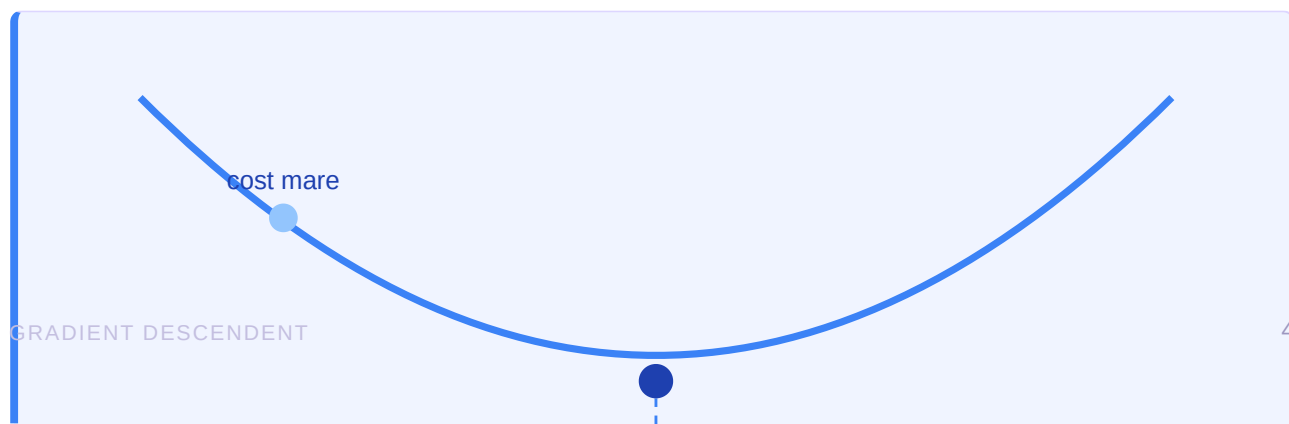
Să o citim în cuvinte. Pentru fiecare dintre cele  $n$  exemple: luăm diferența dintre predicție și valoarea reală, o ridicăm la pătrat (ca să fie mereu pozitivă și ca erorile mari să conteze disproporționat mai mult), apoi facem media tuturor acestor pătrate. Rezultatul  $J$  este un singur număr care rezumă cât de bine se potrivește linia noastră cu datele.

#### DE CE RIDICĂM LA PĂTRAT?

Două motive. Primul: o eroare de  $-5$  și una de  $+5$  sunt la fel de rele, iar pătratul le tratează identic (ambele devin 25). Al doilea: pătratul penalizează puternic erorile mari, împingând modelul să le evite. Bonus tehnic — funcția rezultată este netedă și derivabilă peste tot, exact ce ne trebuie pentru pasul următor.

### Suprafața de cost

Dacă desenăm valoarea lui  $J$  pentru fiecare combinație posibilă de  $w$  și  $b$ , obținem o suprafață în formă de bol (un paraboloid). Acesta este „muntele” din analogia noastră. Punctul cel mai de jos al bolului corespunde celei mai bune linii — perechea  $(w, b)$  pe care o căutăm.



## Gradientul: busola algoritmului

*Gradientul este conceptul care dă numele algoritmului. Vestea bună: în spatele lui stă o idee din liceu — panta.*

### Derivata: panta într-un punct

Pentru o funcție cu o singură variabilă, **derivata** ne spune cât de repede crește sau scade funcția într-un punct. O notăm  $\frac{dJ}{dw}$ . Dacă derivata e pozitivă, funcția urcă spre dreapta; dacă e negativă, funcția coboară spre dreapta.

*Gândește-te la derivată ca la înclinarea terenului exact sub picioarele tale. O pantă abruptă înseamnă o derivată mare; teren plat înseamnă derivată aproape zero. Iar în vale — fundul bolului — terenul e perfect plat, deci derivata este exact zero.*

### Gradientul: panta în mai multe direcții

Modelele reale au mulți parametri, nu doar unul. **Gradientul** este pur și simplu colecția tuturor derivatelor parțiale — câte una pentru fiecare parametru. Îl notăm cu simbolul nabla,  $\nabla J$ :

#### GRADIENTUL — VECTORUL TUTUROR PANTELOR

$$\nabla J = \left( \frac{\partial J}{\partial w_1}, \frac{\partial J}{\partial w_2}, \dots, \frac{\partial J}{\partial w_m} \right)$$

Fiecare componentă  $\frac{\partial J}{\partial w_k}$  răspunde la întrebarea: „dacă modific doar parametrul  $w_k$ , păstrând restul fixe, cât de mult se schimbă costul?” Împreună, aceste componente formează un vector care indică direcția de **creștere cea mai abruptă** a costului.

#### PROPRIETATEA CHEIE A GRADIENTULUI

Gradientul  $\nabla J$  arată mereu spre direcția în care funcția **crește** cel mai rapid. Dar noi vrem să coborâm! Deci ne mișcăm în direcția **opusă** gradientului:  $-\nabla J$ . Acesta este sensul cuvântului „descendent” din numele algoritmului.

### Gradientul pentru regresia liniară

Pentru funcția de cost din Capitolul 2, derivatele parțiale față de cei doi parametri se calculează astfel:

#### DERIVATELE PARȚIALE ALE COSTULUI MSE

$$\frac{\partial J}{\partial w} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_i$$

$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

## Regula de actualizare

Acum avem toate piesele. Le combinăm într-o singură formulă — inima algoritmului gradient descent.

Algoritmul repetă același pas simplu, iar și iar: ia parametri curenți, calculează gradientul, și ajustează parametri în direcția opusă gradientului. În formă matematică:

### REGULA DE ACTUALIZARE A GRADIENT DESCENDENT

$$\theta_{\text{nou}} = \theta_{\text{vechi}} - \alpha \cdot \nabla J(\theta_{\text{vechi}})$$

Să o disecăm bucată cu bucată, pentru că această formulă rezumă tot cursul:

- $\theta$  (theta) reprezintă parametrii modelului — toate numerele pe care le ajustăm (de exemplu  $w$  și  $b$ ).
- $\nabla J(\theta)$  este gradientul — busola care arată direcția de urcare. Semnul minus din fața lui ne face să mergem în jos, nu în sus.
- $\alpha$  (alpha) este **rata de învățare** — un număr mic care controlează cât de mare e pasul. O studiem în detaliu în capitolul următor.
- Semnul = aici înseamnă „atribuire”: valoarea nouă o înlocuiește pe cea veche, și repetăm.

*În termenii muntelui:  $\theta_{\text{vechi}}$  e poziția ta actuală,  $\nabla J$  e direcția de urcare pe care o simți cu piciorul, semnul minus te întoarce spre vale, iar  $\alpha$  decide cât de mare e pasul pe care îl faci. După pas, ești în  $\theta_{\text{nou}}$ , de unde reiei totul.*

### Algoritmul complet, în cinci pași

1. **Inițializează** parametri cu valori aleatoare (un punct de start oarecare pe munte).
2. **Calculează gradientul**  $\nabla J$  în punctul curent — panta locală.
3. **Actualizează** parametri cu regula de mai sus — fă un pas în jos.
4. **Repetă** pașii 2 și 3 de multe ori.
5. **Oprește-te** când gradientul devine aproape zero (terenul e plat) sau după un număr fixat de iterații.

#### CONVERGENȚA

Pe măsură ce ne apropiem de minim, panta scade, deci și gradientul scade. Asta înseamnă că pașii devin automat mai mici aproape de țintă — algoritmul „încetinește” natural când se apropie de soluție. Spunem că algoritmul **converge** atunci când actualizările devin neglijabil de mici.

O singură întrebare rămâne deschisă: cât de mare ar trebui să fie  $\alpha$ ? Răspunsul este surprinzător de important și merită un capitol întreg.

## Rata de învățare

*Rata de învățare  $\alpha$  pare un detaliu minor, dar este probabil cel mai important reglaj din tot algoritmul. Mărimea pasului decide dacă reușești sau eșuezi.*

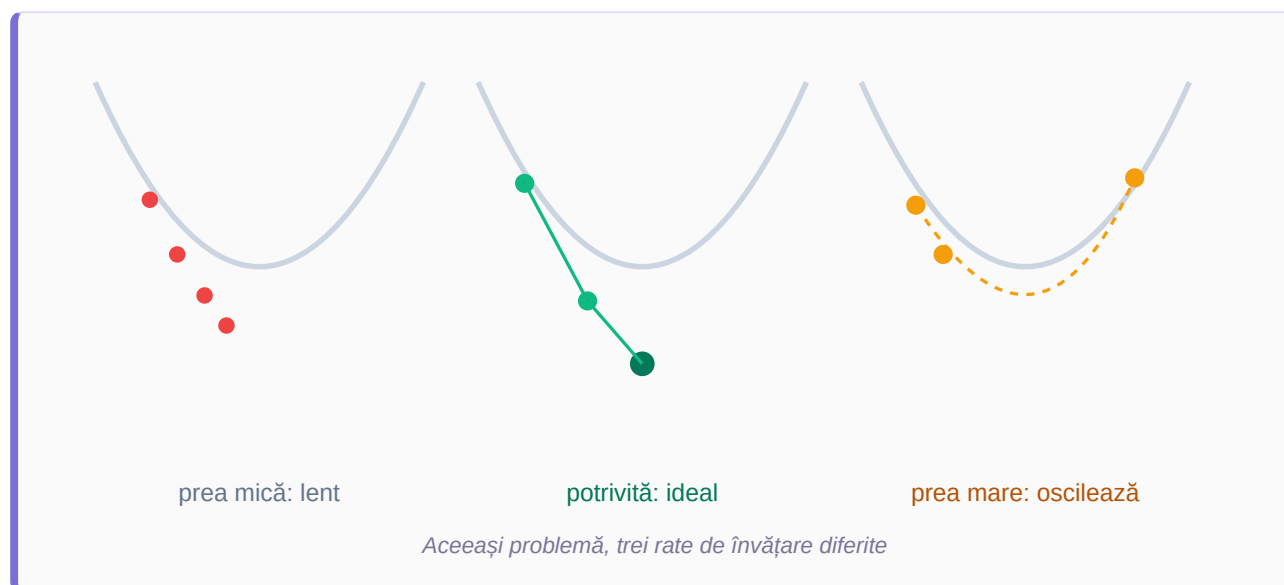
Rata de învățare controlează cât de mult ne mișcăm la fiecare iterație. Alegerea ei e un act de echilibru între viteză și siguranță.

### PREA MICĂ

Pașii sunt minusculi. Algoritmul ajunge în cele din urmă la minim, dar are nevoie de un număr enorm de iterații. Antrenarea devine lentă și costisitoare.

### PREA MARE

Pașii sunt giganți. Algoritmul sare peste minim dintr-o parte în alta, oscilează, și poate chiar *diverge* — costul crește la infinit în loc să scadă.



## Cum alegem o valoare bună?

Nu există o valoare magică universală — depinde de problemă. În practică se pornește adesea de la valori precum **0.1**, **0.01** sau **0.001** și se observă cum evoluează costul. Câteva principii utile:

- Dacă costul **scade lin și constant**, rata este probabil bună.
- Dacă costul **oscilează sau explodează**, rata este prea mare — micșorează-o.
- Dacă costul scade **extrem de încet**, rata este prea mică — mărește-o.

### TEHNICĂ AVANSATĂ: RATE ADAPTIVE

Algoritmii moderni precum **Adam**, **RMSProp** sau **AdaGrad** ajustează automat rata de învățare în timpul antrenării — încep cu pași mai mari și îi micșorează pe parcurs. De aceea sunt folosiți implicit în majoritatea bibliotecilor de deep learning. La bază, însă, toți pornesc de la aceeași idee de gradient descent.

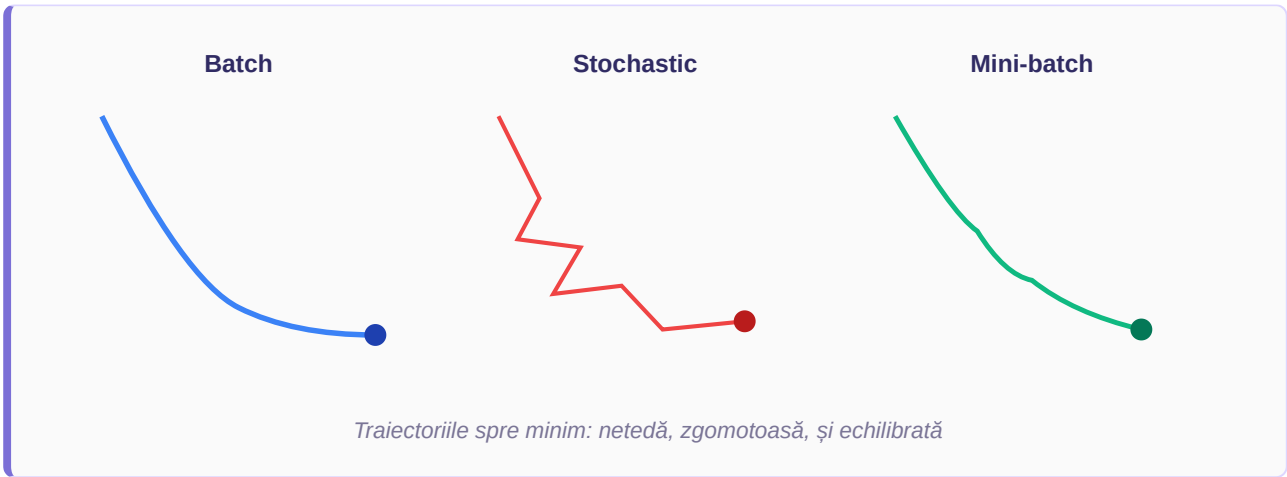
# Variente: Batch, Stochastic, Mini-batch

Până acum am presupus că folosim toate datele la fiecare pas. Când avem milioane de exemple, asta devine impracticabil. Iată cele trei variante care rezolvă problema.

## Întrebarea de fond

Pentru a calcula gradientul exact, trebuie să trecem prin toate exemplele de antrenament. Dar dacă avem 10 milioane de exemple, fiecare pas devine foarte costisitor. Cele trei variante diferă prin **câte exemple folosesc** pentru a estima gradientul la fiecare pas.

| Varianta                   | Câte exemple per pas              | Caracteristici                                                                                                                                   |
|----------------------------|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Batch</b><br>(complet)  | Toate, la fiecare pas             | Gradient exact și traiectorie netedă, dar foarte lent pe seturi mari de date. Necesită multă memorie.                                            |
| <b>Stochastic</b><br>(SGD) | Un singur exemplu, ales aleatoriu | Foarte rapid și folosește puțină memorie. Traiectorie zgomotoasă, „zigzagată”, dar acest zgomot poate ajuta la evadarea din minime locale slabe. |
| <b>Mini-batch</b>          | Un grup mic (ex. 32, 64, 128)     | Compromisul ideal: destul de rapid, destul de stabil. Profită de optimizările hardware. Este standardul în practică.                             |



## Câteva termeni utili

Când lucrezi cu aceste variante vei întâlni doi termeni des folosiți:

- **Epocă** (epoch): o trecere completă prin întregul set de date de antrenament. Antrenarea durează de obicei multe epoci.
- **Dimensiunea batch-ului** (batch size): câte exemple sunt grupate într-un singur pas de actualizare. O alegere comună este între 32 și 256.



## Implementare în Python

*Teoria prinde viață în cod. Iată gradient descent complet pentru regresia liniară, în câteva linii de Python pur — fără biblioteci magice.*

Codul de mai jos găsește cei mai buni parametri  $w$  și  $b$  pentru o relație liniară între  $x$  și  $y$ . Urmărește cum fiecare linie corespunde direct formulelor din capitolele anterioare.

```
import numpy as np

# Datele de antrenament: x = intrare, y = ieșire reală
x = np.array([1, 2, 3, 4, 5])
y = np.array([3, 5, 7, 9, 11]) # relația reală: y = 2x + 1

# 1. Inițializăm parametrii cu valori arbitrare
w, b = 0.0, 0.0
alpha = 0.01 # rata de învățare
epoci = 1000 # câte iterații facem
n = len(x)

for i in range(epoci):
    # 2. Predicția curentă:  $\hat{y} = w \cdot x + b$ 
    y_pred = w * x + b

    # Eroarea: diferența dintre predicție și real
    eroare = y_pred - y

    # 3. Calculăm gradientul (derivatele parțiale)
    grad_w = (2/n) * np.sum(eroare * x)
    grad_b = (2/n) * np.sum(eroare)

    # 4. Actualizăm parametrii:  $\theta = \theta - \alpha \cdot \nabla J$ 
    w = w - alpha * grad_w
    b = b - alpha * grad_b

print(f"w = {w:.3f}, b = {b:.3f}")
# Rezultat: w ≈ 2.000, b ≈ 1.000 → a învățat y = 2x + 1
```

### Ce se întâmplă, linie cu linie

- Pornim cu  $w = 0$  și  $b = 0$  — un model complet greșit la început.
- La fiecare epocă calculăm predicțiile și erorile pentru toate exemplele simultan (datorită operațiilor vectoriale NumPy).
- Calculăm gradientul folosind exact formulele din Capitolul 3.
- Aplicăm regula de actualizare din Capitolul 4, mutând  $w$  și  $b$  puțin mai aproape de soluție.
- După 1000 de repetiții, parametrii converg spre valorile reale:  $w = 2$ ,  $b = 1$ .

## Capcane, sfaturi și recapitulare

*Ai parcurs întregul drum, de la intuiție la cod. Iată ce să eviți, ce să reții, și unde să mergi mai departe.*

### Capcane frecvente

- **Minime locale.** Pe suprafețe complexe (rețele neuronale), pot exista mai multe „văi”. Algoritmul s-ar putea opri într-una care nu e cea mai adâncă. Zgomotul din SGD și optimizatorii moderni ajută la atenuarea problemei.
- **Date nenormalizate.** Dacă parametrii au scări foarte diferite, suprafața de cost devine alungită, iar coborârea ineficientă. Soluția: normalizează datele de intrare înainte de antrenare.
- **Rată de învățare greșită.** Cea mai comună cauză a eșecului. Dacă antrenarea „explodează” sau stagnează, primul lucru de ajustat este  $\alpha$ .
- **Prea multe sau prea puține iterații.** Prea puține: modelul nu apucă să învețe. Prea multe: pierdere de timp după ce a convers deja.

#### RECAPITULARE ÎN TREI PROPOZIȚII

Gradient descent caută parametrii care minimizează o funcție de cost, mergând repetat „la vale” pe suprafața erorii.

La fiecare pas, gradientul indică direcția de urcare, iar noi ne mișcăm în sens opus, cu o mărime dată de rata de învățare.

Formula centrală  $\theta_{\text{nou}} = \theta_{\text{vechi}} - \alpha \nabla J$  rezumă tot algoritmul, de la regresie liniară la rețele neuronale gigantice.

### De ce contează

Gradient descent este motorul care antrenează aproape fiecare model modern de învățare automată — de la simple regresii până la modelele lingvistice cu miliarde de parametri. Înțelegerea lui îți dă cheia pentru a pricepe cum „învață” de fapt inteligența artificială: nu prin magie, ci prin pași mici și repetați de îmbunătățire.

### Unde mergi mai departe

- **Backpropagation** — algoritmul care calculează eficient gradientii în rețele neuronale cu multe straturi.
- **Optimizatori avansați** — studiază Adam, RMSProp și momentum pentru a vedea cum se accelerează convergența.
- **Regularizare** — tehnici (L1, L2, dropout) care modifică funcția de cost pentru a evita supra-antrenarea.
- **Practică** — implementează gradient descent pentru regresie logistică, apoi pentru o mică rețea neuronală.

#### FELICITĂRI

Ai înțeles unul dintre cele mai importante concepte din inteligența artificială. Următorul pas e să-l pui în practică pe date reale — acolo intuiția devine măiestrie.